Semantic Layer – Test Cases

Tester : Rabea Adel

Data : 15/9/2026

Upload Semantic Layer

Id	TC-SL-upload-001
Test scenario	Verify that the user can successfully upload a Semantic Layer with all supported data source files (Schema, Documentation, Glossary, and Sample Data). (user ROLE = admin)
Expected Result	The API should return 200 OK create a new Semantic Layer, save all uploaded files successfully, and return valid IDs for the Semantic Layer and all uploaded source files.
Actual Result	API returned 200 OK; Semantic Layer upload completed successfully with the supplied source files.
Status	PASS

Id	TC-SL-upload-002
Test scenario	Verify that the user can successfully upload a Semantic Layer with only the required Schema file while omitting Documentation, Glossary, and Sample Data files. (user ROLE = admin)
Expected Result	The API should return 200 OK, create the Semantic Layer successfully, upload the Schema file ,return HasDocumentation = false, HasGlossary = false, and HasSampleData = false.
Actual Result	API returned 200 OK; HasDocumentation=false, HasGlossary=false, and HasSampleData=false.
Status	PASS

Id	TC-SL-upload-003
Test scenario	Verify that the API rejects the upload request when the required Schema file is not provided. (no upload schema file) (user ROLE = admin)
Expected Result	The API should return 400 Bad Request and schema file is required
Actual Result	API returned 400 Bad Request; schema file is required.
Status	PASS

Id	TC-SL-upload-004
Test scenario	Verify that the API rejects the upload request with all empty data (user ROLE = admin)
Expected Result	The API should return 400 Bad Request and schema ,name , description , schemFile is required
Actual Result	API returned 400 Bad Request; required name, description, and schema fields were rejected when empty.
Status	PASS

Get Source File

Id	TC-SL-get-001
Test scenario	Verify that an authenticated user can retrieve a previously uploaded Semantic Layer source file using a valid fileId (user ROLE = admin)
Expected Result	API returns 200 OK and the response contains the requested file information and content.
Actual Result	API returns 200 OK and the response contains the requested file information and content.
Status	pass

Id	TC-SL-get-002
Test scenario	Verify that an authenticated user can retrieve a previously uploaded Semantic Layer source file using a invalid fileId (user ROLE = admin)
Expected Result	API returns 400 Bad Req
Actual Result	400
Status	PASS

Id	TC-SL-get-003
Test scenario	Verify that an authenticated user can retrieve a previously uploaded Semantic Layer source file using a valid fileId (user ROLE = user)
Expected Result	The API should return 401 unauth
Actual Result	HTTP 404. Non-existing file ID 00000000-0000-0000-0000-000000000000 returned: "File not found."
Status	PASS

Get File Content

Id	TC-SL-content-001
Test scenario	Verify that an authenticated admin can download the binary

	content of a previously uploaded source file. Endpoint: GET /api/v1/semantic-layer/files/{fileId}. (Use the /content route; user ROLE = admin)
Expected Result	API should return 200 OK with the file bytes, the correct Content-Type based on file extension, the original file name, and range processing enabled.
Actual Result	API return 200 OK with the file bytes, the correct Content-Type based on file extension, the original file name, and range processing enabled.
Status	PASS

Id	TC-SL-content-002
Test scenario	Verify that GET /api/v1/semantic-layer/files/{fileId}/content rejects a non-existing fileId. (user ROLE = admin)
Expected Result	API should return 404 Not Found with status = Failed, errorCode = NOT_FOUND, and a file-not-found message.
Actual Result	HTTP 404. Non-existing file ID 00000000-0000-0000-0000-000000000000 returned the expected file-not-found response.
Status	PASS

Revision Retrieval

Id	TC-SL-revision-get-001
Test scenario	Verify that an authenticated admin can retrieve an existing Semantic Layer revision using a valid revisionId. Endpoint: GET /api/v1/semantic-layer/revisions/{revisionId}.
Expected Result	API should return 200 OK and include SemanticLayerId,

	RevisionId, Status, Version, BuildTimestamp, LastRegenerationType, Content, and CreatedAt.
Actual Result	HTTP 200. Active revision 1c9ce605-041b-4dd2-87da-57deb4d85f48 returned with status Approved, version v1.0, 7 entities, 6 relationships, 3 measures, and validationIssues empty.
Status	PASS

Id	TC-SL-revision-get-002
Test scenario	Verify that GET /api/v1/semantic-layer/revisions/{revisionId} rejects a non-existing revisionId. (user ROLE = admin)
Expected Result	API should return 404 Not Found with errorCode = NOT_FOUND and message = "Revision not found."
Actual Result	HTTP 404. Non-existing revision ID returned: "Revision not found."
Status	PASS

Generate Draft

Id	TC-SL-draft-001
Test scenario	Verify that an authenticated admin can generate a Semantic Layer draft for an existing layer with a valid SemanticLayerId and available RLS policy source. Endpoint: POST /api/v1/semantic-layer/generate-draft. Test Data: TriggerType = FullRebuild; SourceFileIds contains the RLS policy file ID.
Expected Result	API should return 200 OK and create a new revision with Status = DraftGenerated, a valid RevisionId, the supplied SemanticLayerId, BuildTimestamp, LastRegenerationType, and RegeneratedObjectsCount. The created revision should initially be PendingReview.
Actual Result	API return 200 OK and create a new revision with Status = DraftGenerated, a valid RevisionId, the supplied SemanticLayerId, BuildTimestamp, LastRegenerationType, and RegeneratedObjectsCount. The created revision should initially be PendingReview.
Status	PASS

Review Revision

Id	TC-SL-review-001
Test scenario	Verify that an authenticated admin can reject an existing revision. Endpoint: POST /api/v1/semantic-layer/review. Test Data: valid SemanticLayerId, valid RevisionId, Decision = "Reject", Comments = "Reject for QA validation."
Expected Result	API should return 200 OK; revision status should become Rejected and the response should contain SemanticLayerId, RevisionId, Status = Rejected, Comments, RejectedBy, and RejectedAt.
Actual Result	API return 200 OK

Status	PASS

Id	TC-SL-review-003
Test scenario	Verify that review rejects invalid SemanticLayerId or RevisionId formats.
Expected Result	API should return 400 Bad Request with errorCode = BUSINESS_ERROR and message = "Invalid Id format."
Actual Result	HTTP 400. BUSINESS_ERROR: "Invalid Id format."
Status	PASS

Submit Revision

Id	TC-SL-submit-001
Test scenario	Verify that an authenticated admin can submit a valid revision for AI validation. Endpoint: POST /api/v1/semantic-layer/revisions/{revisionId}/submit.
Expected Result	API should return 200 OK with Status = Submitted, the correct SemanticLayerId and RevisionId, and message indicating that the revision was submitted and validated successfully by AI Runtime.
Actual Result	return 200 OK
Status	PASS

Semantic Layer Status

Id	TC-SL-status-003
Test scenario	Verify that GET /api/v1/semantic-layer/status without the optional id parameter resolves the active Semantic Layer.
Expected Result	If an active layer exists, API should return 200 OK with its status. If no active layer exists, API should return 404 Not Found with message = "No active Semantic Layer found."
Actual Result	HTTP 200. Active Semantic Layer bfeb8e3f-2e93-4d06-9ca3-cece0ecfcc32 returned with Approved revision 1c9ce605-041b-4dd2-87da-57deb4d85f48.
Status	PASS

Active Schema

Id	TC-SL-active-schema-001
Test scenario	Verify that GET /api/v1/semantic-layer/revisions/active/schema returns the physical schema of the approved revision belonging to the active Semantic Layer.
Expected Result	API should return 200 OK and include SemanticLayerId, RevisionId, Status = Approved, and Schema as valid JSON containing the physical schema.
Actual Result	HTTP 200. Active approved physical schema returned; 7 tables and 6 relationships were present.
Status	PASS

List / Detail Semantic Layers

Id	TC-SL-list-001

Test scenario	Verify that GET /api/v1/semantic-layer returns the list of Semantic Layers for an authenticated admin.
Expected Result	API should return 200 OK with a JSON array. Each item should include SemanticLayerId, Name, DatabaseName, Description, IsActive, and HasApprovedRevision.
Actual Result	HTTP 200. 28 Semantic Layers returned; 1 active, 21 with approved revisions, and 7 without an approved revision.
Status	PASS

Id	TC-SL-list-002
Test scenario	Verify that GET /api/v1/semantic-layer?id={layerId} filters the result to the requested Semantic Layer.
Expected Result	API should return 200 OK. For an existing id, the response should contain the matching layer only; for a non-existing id, the expected response is an empty array because the service returns the query result without a not-found error.
Actual Result	HTTP 200. Filtering by layerId bfeb8e3f-2e93-4d06-9ca3-cece0ecfcc32 returned the active layer.
Status	PASS

Toggle Table

Id	TC-SL-toggle-table-001
Test scenario	Verify that an authenticated admin can enable/disable access for an existing table. Endpoint: PATCH /api/v1/semantic-layer/{layerId}/tables/{tableName}/toggle. Body: true or false.
Expected Result	API should return 200 OK with message = "Table '<tableName>' access set to <isAllowed>" and the table permission should be updated.
Actual Result	HTTP 200. sysdiagrams access was set to false successfully.
Status	PASS

Get Tables

Id	TC-SL-tables-001
Test scenario	Verify that GET /api/v1/semantic-layer/{layerId}/tables returns the allowed-table configuration for a Semantic Layer.
Expected Result	API should return 200 OK with SemanticLayerId and a Tables collection containing TableName and IsAllowed for each configured table.
Actual Result	HTTP 200. 8 tables were returned. Subsequent verification showed sysdiagrams IsAllowed=false.
Status	PASS

Id	TC-SL-tables-003
Test scenario	Verify that the table permission change persists after toggling an existing table.
Expected Result	A subsequent GET should show the selected table with IsAllowed=false after disabling it.
Actual Result	HTTP 200. Subsequent GET showed sysdiagrams with IsAllowed=false after the toggle.
Status	PASS

Sync Metadata

Id	TC-SL-sync-001
Test scenario	Verify that an authenticated admin can synchronize database metadata. Endpoint: POST /api/v1/semantic-layer/{layerId}/sync-metadata.
Expected Result	API should return 200 OK with SemanticLayerId, DatabaseName, SyncedAtUtc, and Metadata. AllowedTables should be synchronized with the target database metadata.
Actual Result	Initial empty-body request returned 411 due to missing Content-Length; retry with Content-Length: 0 returned HTTP 200 and metadata was synchronized.
Status	PASS

RLS Policy

Id	TC-SL-rls-get-001
Test scenario	Verify that an authenticated admin can retrieve the stored RLS policy. Endpoint: GET /api/v1/semantic-layer/{layerId}/rls-policy.
Expected Result	API should return 200 OK with the stored RLS policy JSON. If the layer exists but no policy is stored, the response should indicate enabled = false with an empty rules collection.
Actual Result	HTTP 200. RLS policy returned enabled=true, ScopeParameter=@UserBranchId, UserValueField=BranchId, BranchMapping branches/branch_id/branch_name, and 6 rules.
Status	PASS

Id	TC-SL-rls-upload-001
Test scenario	Verify that an authenticated admin can upload a valid RLS policy JSON file. Endpoint: PUT /api/v1/semantic-layer/{layerId}/rls-policy/file. Test Data: valid JSON containing ScopeParameter, BranchMapping { Table, IdColumn, optional NameColumn }, and Rules with Table and ScopeColumn.
Expected Result	API should return 200 OK with status = Success and message = "RLS policy file uploaded and saved successfully."
Actual Result	HTTP 200. Valid RLS policy uploaded and saved successfully.
Status	PASS

Id	TC-SL-rls-upload-003
Test scenario	Verify that RLS policy upload rejects malformed JSON.
Expected Result	API should return 400 Bad Request with errorCode = RLS_POLICY_ERROR and message = "The uploaded file contains invalid JSON."
Actual Result	HTTP 400. RLS_POLICY_ERROR: "The uploaded file contains invalid JSON."
Status	PASS

Id	TC-SL-rls-upload-004
Test scenario	Verify that RLS policy upload rejects a valid JSON file with an invalid RLS structure.
Expected Result	API should return 400 Bad Request with errorCode = RLS_POLICY_ERROR and a validation message identifying the invalid BranchMapping structure.
Actual Result	HTTP 400. RLS_POLICY_ERROR: "BranchMapping must contain a valid table and idColumn, plus an optional valid nameColumn."
Status	PASS

User Table Permissions

Id	TC-SL-permissions-001
Test scenario	Verify that an authenticated admin can retrieve user table permissions. Endpoint: GET /api/v1/semantic-layer/{layerId}/users/table-permissions.
Expected Result	API should return 200 OK with SemanticLayerId, TableNames, and Users. Each user should include UserId, Email, DisplayName, and per-table boolean permissions.
Actual Result	HTTP 200. User table permissions returned 8 users and 8 tables; permissions were initially enabled.
Status	PASS

User Table Permission

Id	TC-SL-user-permission-001
Test scenario	Verify that an authenticated admin can set a user's table permission. Endpoint: PATCH /api/v1/semantic-layer/{layerId}/users/table-permission. Query: email=<valid user>, tableName=<existing table>. Body: true.
Expected Result	API should return 200 OK with a message confirming the selected table access for the selected user.
Actual Result	HTTP 200. ahmed.user@gmail.com accounts permission was set to false successfully.
Status	PASS